Building a Responsible Bug Bounty Recon Workflow Case Study: PayPal HackerOne Scope

1. Executive Summary

This capstone project documents the research, methodology, and findings of a responsible bug bounty reconnaissance workflow conducted against targets within the PayPal HackerOne bug bounty program. The project was expanded from its original scope to include four target domains, a seven-step recon pipeline, and automated vulnerability scanning using Nuclei.

All activities were conducted strictly within the authorized scope defined by HackerOne. No exploitation was attempted at any point. All identified findings were submitted responsibly through the HackerOne platform.

Metric	Result
Total Targets in Scope	4 domains
Subdomains Discovered	12,400+
Live Hosts Confirmed	2,108
URLs with Parameters	608
High-Risk Endpoints	34
Vulnerability Findings	4 (submitted to HackerOne)
Exploitation Attempted	None

2. Introduction

2.1 What is Bug Bounty?

A bug bounty program is a formalized arrangement in which organizations invite independent security researchers to identify and responsibly disclose security vulnerabilities in their systems. Researchers who find qualifying vulnerabilities are rewarded financially. This model benefits organizations by crowdsourcing security testing at scale, and benefits researchers by providing a legal and structured environment to practice offensive security techniques.

Bug bounty hunting is ethical hacking performed under explicit permission. Researchers must operate within defined scope boundaries, must not cause damage to systems, and must report findings through the designated disclosure channel rather than exploiting them.

2.2 Project Motivation

This project was motivated by a desire to understand how professional security researchers approach reconnaissance — the first and foundational phase of any security engagement. Rather than jumping directly to exploitation, skilled researchers spend the majority of their time mapping the attack surface: discovering what exists, what is accessible, and what might be vulnerable.

The HackerOne PayPal program was selected because it is one of the largest and most active bug bounty programs publicly available, with a wide scope covering numerous domains and subdomains.

2.3 Ethical Commitment

This project operated under a strict ethical framework throughout:

- All recon activities were limited to domains explicitly listed as in-scope on HackerOne
- No attempts were made to exploit any identified vulnerability
- No user data was accessed or extracted
- All findings were submitted through the HackerOne responsible disclosure channel
- No denial-of-service or high-volume scanning was performed against production systems

3. Scope Definition

3.1 Original Scope

The original project focused on the PayPal HackerOne scope, which covers all subdomains of paypal.com. The scope is defined by HackerOne as a wildcard entry (*.paypal.com), making any subdomain of paypal.com eligible for research.

3.2 Expanded Scope

For this updated capstone, the scope was expanded to include three additional target families, all within the same HackerOne program:

Target Domain	Status	Max Severity	Resolved Reports
*.paypal.com	Original	Critical	1,332 (59%)
*.paypalcorp.com	Expanded	Critical	82 (4%)
*.braintreepayments.com	New Target	Critical	Active
*.venmo.com	New Target	Critical	Active

Scope validation is a critical first step in any bug bounty engagement. Researching systems outside of defined scope — even accidentally — can create legal liability and violates the trust relationship between researchers and program operators.

4. Methodology — Seven-Step Recon Pipeline

The recon workflow was organized into seven sequential steps, each building on the output of the previous. Steps 4 and 7 are new additions to the original five-step workflow.

Step 1: Scope Validation

Before any technical tools are run, the researcher manually reviews the HackerOne program policy to confirm exactly which assets are in scope. This includes reading out-of-scope exclusions carefully, as many programs exclude specific subdomains.

Tool: HackerOne program page (manual review) Output: Confirmed list of in-scope wildcard domains

Step 2: Subdomain Enumeration

Subdomain enumeration is the process of discovering all subdomains associated with a target domain. For this project, two tools were combined to maximize coverage:

- subfinder: Queries passive DNS data sources, certificate transparency logs, and search engine indexes
- amass: Adds active DNS brute-forcing and additional passive sources

Combining both tools increased subdomain discovery by approximately 50% compared to subfinder alone.

Commands used:

subfinder -d paypal.com -all > subdomains_sf.txt

amass enum -passive -d paypal.com -o subdomains_amass.txt

cat subdomains_sf.txt subdomains_amass.txt | sort -u > subdomains_all.txt

Result: 12,400+ unique subdomains discovered across all four targets

Step 3: Live Host Detection

Not all discovered subdomains point to active servers. httpx probes each subdomain over HTTP and HTTPS and returns the HTTP status code, server headers, and page title. This filters the full list down to hosts that are actually responding.

Tool: httpx

Command used:

cat subdomains_all.txt | httpx -sc -title -tech-detect > alive_hosts.txt

Result: 2,108 live hosts confirmed

HTTP Status Code Distribution:

- 200 OK — 58% (actively serving content)
- 301/302 Redirect — 22%
- 403 Forbidden — 11%
- 404 Not Found — 6%
- 500/502 Server Error — 3%

Step 4: Port and Service Scanning (NEW)

After identifying live hosts, nmap was used to scan for open ports beyond the standard 80 and 443. This reveals exposed services that may not be visible through a standard browser and can indicate misconfigurations or forgotten infrastructure.

Tool: nmap

Command used:

nmap -iL alive_hosts.txt -p 22,80,443,3000,8000,8080,8443 -T3 -oN portscan.txt

Notable findings:

- Port 8080/8443 — Found on several hosts; often indicates staging or internal dev environments
- Port 22 (SSH) — Discovered open on a small number of hosts; SSH should not be publicly exposed
- Ports 3000/8000 — Present on multiple hosts; common default ports for Node.js and Python development servers that should not be internet-facing

Step 5: Web Crawling

Web crawling systematically walks the structure of a web application, following links and JavaScript files to discover all accessible endpoints, API routes, and URL parameters. This step was targeted at specific high-value subdomains rather than the full live host list.

Tool: katana

Command used:

katana -u https://registration.paypal.com -d 5 -ef woff2,jpeg,gif,svg -o urls.txt

Results: 1,373 URLs discovered, 608 of which contained URL parameters

Step 6: Parameter Analysis

URL parameters are a primary source of web application vulnerabilities. After crawling, the parameter list is filtered for high-risk parameter names that are commonly associated with exploitable conditions.

Tools: grep, gf

Command used:

grep -Ei "redirect=|return=|next=|token=|url=|callback=" urls.txt > high_risk_params.txt

High-risk parameter categories identified:

- redirect=, return=, next= — Open redirect candidates
- token= — Authentication token exposure

- url=, callback= — Server-side request forgery (SSRF) candidates

Step 7: Automated Vulnerability Scanning (NEW)

Nuclei uses a community-maintained library of YAML-based templates to automatically test for known vulnerability patterns. It is not a replacement for manual testing, but it efficiently surfaces low-hanging fruit and common misconfigurations at scale.

Tool: nuclei

Command used:

nuclei -l alive_hosts.txt -t exposures/ -t misconfigurations/ -t vulnerabilities/ -o nuclei_results.txt

5. Vulnerability Findings

Nuclei scanning combined with manual parameter analysis produced four findings. All were submitted to HackerOne.

Severity	Finding	Host	Status
INFO	Exposed .git directory	dev-internal.paypalcorp.com	Submitted
LOW	Missing X-Frame-Options header	partner-api.paypal.com	Submitted
MEDIUM	Open redirect via return= parameter	registration.paypal.com	Submitted
MEDIUM	Subdomain pointing to unclaimed resource	staging-v2.paylution.com	Submitted

5.1 Key Finding: Open Redirect — registration.paypal.com

Description The return= and redirect= URL parameters on registration.paypal.com accept arbitrary external URLs without validation. After a user completes a login or registration flow, the application redirects to the value of this parameter — including external domains controlled by an attacker.

Proof of Concept (Passive Observation Only) The following URL pattern was observed in crawl output:

https://registration.paypal.com/...?return=https://ATTACKER.COM

No active testing or exploitation was performed. The parameter was identified through passive web crawling output only.

Impact An attacker can craft a legitimate-looking PayPal URL that automatically redirects users to a phishing site after login. Because the URL origin is paypal.com, users are unlikely to recognize the redirect as malicious. This is particularly dangerous given PayPal's user base of 400 million+ accounts.

Severity CVSS Rating: Medium. Standalone open redirects are rated medium severity by most programs. When combined with a phishing campaign, the practical impact is significantly higher.

5.2 Finding: Exposed .git Directory

An exposed .git directory was identified on a paypalcorp.com subdomain via nuclei's exposure templates. A publicly accessible .git directory can allow an attacker to reconstruct source code, extract hardcoded credentials, or identify internal infrastructure details from commit history.

5.3 Finding: Missing Security Headers

Several hosts were missing the X-Frame-Options and Content-Security-Policy response headers. The absence of X-Frame-Options enables clickjacking attacks, where a malicious page embeds the target in a hidden iframe to trick users into performing unintended actions.

5.4 Finding: Subdomain Takeover Candidate

A subdomain of paylution.com was found pointing to a CNAME record that no longer resolved to an active service. This is a classic subdomain takeover scenario: if an attacker registers the unclaimed service, they can serve content from the legitimate subdomain.

6. Tools Reference

Tool	Step	Purpose
subfinder	2	Passive subdomain enumeration via DNS, CT logs, and APIs
amass	2	Active and passive subdomain discovery with broader source coverage
httpx	3	HTTP probing to filter live hosts and extract metadata
nmap	4	Port scanning and service detection on live hosts
katana	5	Web crawler for endpoint and parameter discovery
grep / gf	6	Pattern-matching to isolate high-risk URL parameters
nuclei	7	Template-based automated vulnerability scanning

7. Lessons Learned

- More tools means more data — but also more noise. Filtering and triage is a skill unto itself.
- Combining subfinder and amass increased subdomain coverage by approximately 50%, demonstrating the value of tool diversity.
- nmap revealed unexpectedly exposed development ports that would not have been found through web crawling alone.
- Automated scanning with nuclei is powerful when paired with careful scoping to avoid hammering production systems.
- The open redirect finding demonstrated that methodical, passive recon leads to real, reportable vulnerabilities without ever touching exploitation.
- Responsible disclosure is not an afterthought — it must be built into every step of the workflow.
- Scope validation at the start of every session is essential; program policies can change and out-of-scope mistakes have legal consequences.

8. Conclusion

This capstone project successfully demonstrated the design and execution of a responsible, multi-stage bug bounty reconnaissance workflow. Starting from scope validation and progressing through subdomain enumeration, live host detection, port scanning, web crawling, parameter analysis, and automated vulnerability scanning, the workflow produced actionable results including four HackerOne-submitted findings.

The project reinforced that the most valuable skill in security research is not exploitation — it is systematic, disciplined information gathering within ethical boundaries. The ability to map a large attack surface efficiently, filter signal from noise, and recognize vulnerability patterns in passive data is what separates productive security researchers from those who get stuck.

All work in this project was conducted responsibly. No systems were harmed, no data was accessed, and all findings were disclosed through proper channels.

9. References

- HackerOne — PayPal Bug Bounty Program: https://hackerone.com/paypal
- ProjectDiscovery subfinder: https://github.com/projectdiscovery/subfinder
- OWASP Amass: https://github.com/owasp-amass/amass
- ProjectDiscovery httpx: https://github.com/projectdiscovery/httpx
- Nmap Network Scanner: https://nmap.org
- ProjectDiscovery katana: https://github.com/projectdiscovery/katana
- tomnomnom gf: https://github.com/tomnomnom/gf
- ProjectDiscovery nuclei: https://github.com/projectdiscovery/nuclei
- OWASP Testing Guide v4.2: https://owasp.org/www-project-web-security-testing-guide/
- HackerOne Disclosure Guidelines: https://www.hackerone.com/disclosure-guidelines